

暗室藏书 · PROJECT REVIEW

项目复盘

从 5 月构想到可验证的公开基线
记录业务、架构、测试、文档与发布治理

一、复盘前提

本次复盘不把项目包装成从第一天就设计完毕的产品。想法形成于 2026 年 5 月，7 月短学期成为落地窗口；课程阶段先形成可演示版本，课程提交后继续补齐跨层契约、业务闭环、版本迁移、真实联调、并发与浏览器验证，最后统一公开材料。

教师发放的教学底座只提供过普通登录、用户和公告等早期条件。归档对比显示，当前版本的技术栈、包名、十九张业务表、5 个角色码、6 个固定权限身份、业务闭环和界面均已形成独立实现。因此，本文件只把底座视为课程阶段背景，不把它写成当前工程的主体。

复盘结论：项目已成为一套可运行、可解释、可维护的模块化单体；结论以代码、测试、数据库检查、浏览器诊断和最终材料相互印证的事实为准。

二、重构做了什么

- 补齐借阅、归还、续借、预约、罚款和库存恢复的状态闭环。
- 补齐书评、点赞、回复、举报、内容审核、留言和附件治理。
- 补齐采购需求、认领、物流、到馆、入库和库存补充的跨角色流程。
- 重构 JWT、角色边界、数据范围、文件生命周期、通知补偿和操作审计。
- 将 Vue 2 遗留结构统一迁移到 Vue 3.5 + Vite 8.1.5。
- 将 Spring Boot 2.7 迁移到 Spring Boot 3.5，并完成 Jakarta 与测试适配。
- 重写演示账号、书目、封面、头像、评论、公告和截图为项目专用虚构数据。

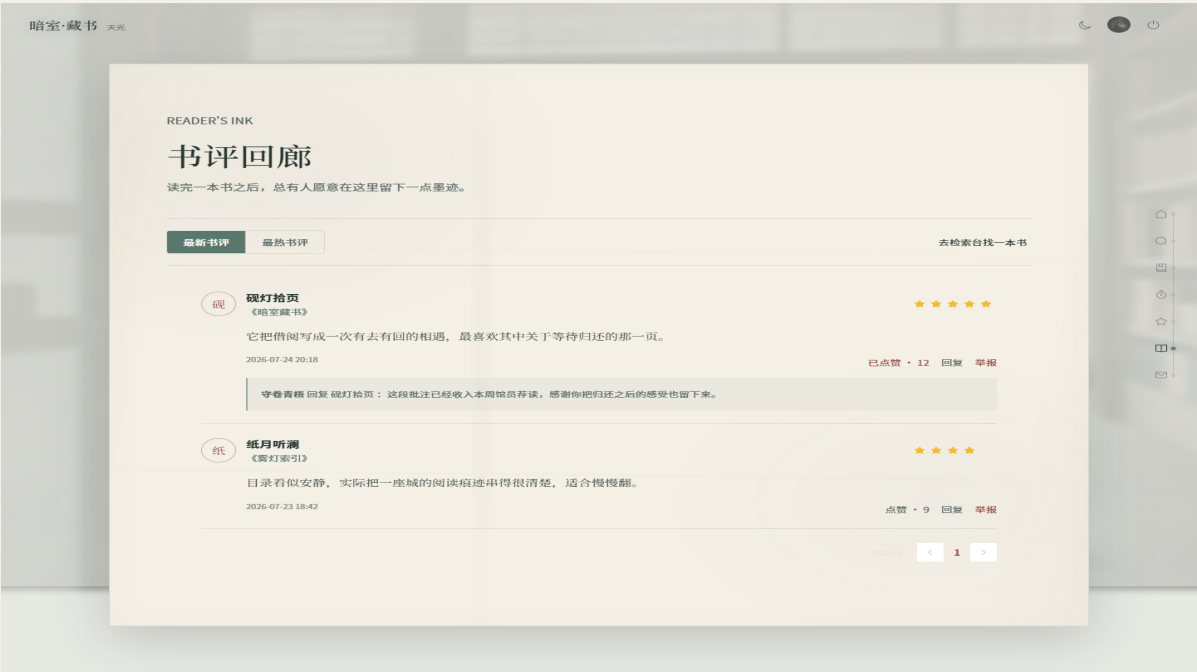


图 1 书评、回复与治理页面，展示内容闭环。

三、验证顺序

顺序	检查	目的
1	基线保留	保留原始交付物和变更证据，避免把旧问题误判为新问题
2	后端与前端自动测试	验证局部规则和稳定回归
3	真实 API 与六个权限身份	验证服务、数据库和权限真实联动
4	并发与数据库一致性	验证库存、状态、幂等和约束
5	浏览器 / F12 诊断	验证 Console、Network、页面和溢出
6	代码与文档审查	统一架构叙事，删除四不像逻辑和陈旧说明
7	PPT、PDF、docs、Git、GitHub	让公开材料与当前代码保持同一事实基线

四、真实全链路结果

5 个角色码对应的 6 个固定权限身份均通过真实登录接口进入当前服务，并完成跨角色业务操作：普通管理员与馆务协调员分别验证日常馆务和跨管理员协调边界，读者负责借还和互动，采购员负责采购，物流员负责运输和入库，超级管理员负责全局授权和审计。全流程记录 78 次真实 API 响应。

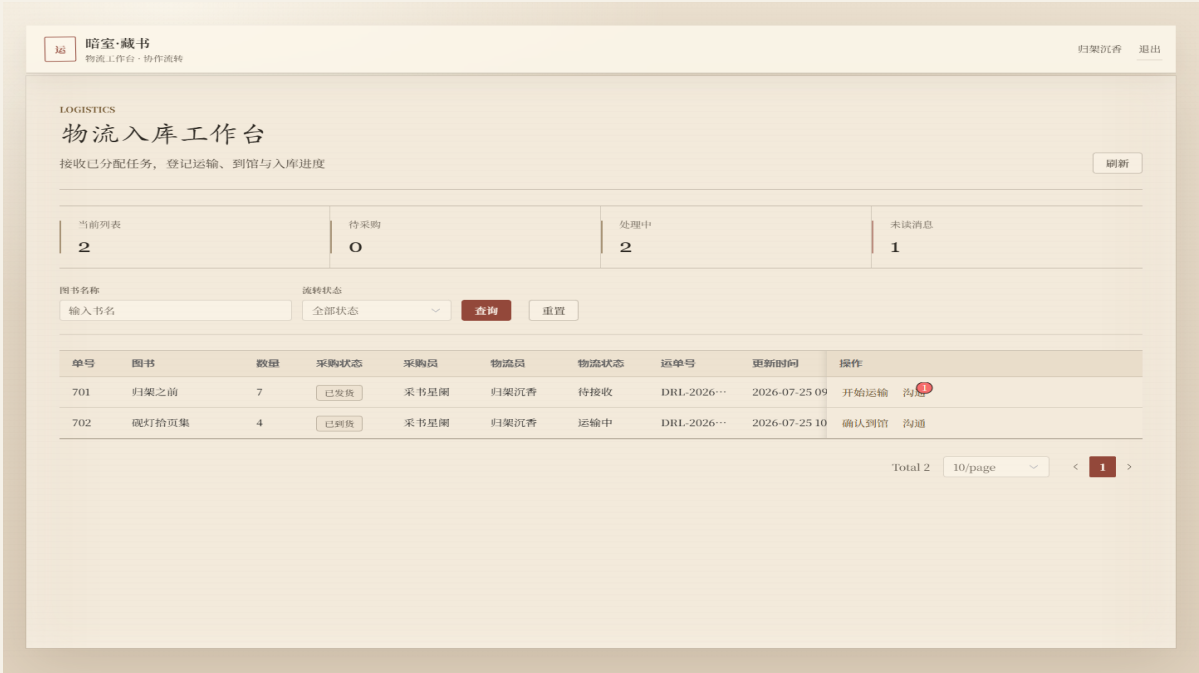


图 2 物流工作台，展示采购到入库链路的实际界面。

链路	验证内容	结果
读者流通	查询、借阅、归还、续借、预约、收藏、互动	通过
权限治理	超级管理员、馆务协调员、普通管理员、读者、采购员、物流员边界	通过
采购协作	创建、认领、推进、分配物流	通过
物流入库	运输、到馆、入库、库存补充	通过

链路	验证内容	结果
幂等性	重复归还、重复预约取消、重复入库	通过

五、并发与数据库一致性

三个后端实例共享 MySQL，按突发量 96、128、160 分三批执行。每批包含 8 个一致性场景，三批共 1,986 次场景请求，最大场景 P95 为 461 ms。库存为 1 的并发借阅只允许一个请求成功；条件更新、事务、租约和唯一约束共同防止库存为负、活动借阅重复、通知重复处理和重复入库。

数据库验证覆盖 27 条外键关系，孤儿记录为 0；库存范围、活动借阅唯一性、活动预约唯一性、采购入库幂等和文件引用不变量均为 0 违规。MySQL 是最终事实来源，Redis 只做缓存和风控增强，RabbitMQ 只做事件和通知增强。



图 3 统计看板，展示当前管理端数据呈现。

六、浏览器与 F12

浏览器诊断覆盖 116 个路由、414 次 API 响应和 5,678 次总网络响应，包含桌面和移动视口、6 个固定权限身份登录与路由、主要交互页、图表画布和动态依赖加载。另检查 116 个页面和 21 个关键弹层。Console 错误与警告、页面异常、失败请求、网络错误、横向溢出和非预期越界均为 0。

浏览器诊断项	结果
Console 错误 / 警告	0
页面异常	0
失败请求 / 网络错误	0
横向溢出路由	0
桌面与移动视口	均通过

七、文档与视觉复盘

旧材料的问题不只是版本号过时，还包括项目背景、角色命名和测试证据不一致。此次处理把 README、NOTICE、CONTRIBUTING、SECURITY、docs、PPT 和 PDF 对齐到同一基线：5 月形成想法，7 月借课程落地，课后继续工程化，Git 在中途加入。

PPT 保留原 18 页模板、母版、布局和叙事骨架，只替换当前截图、版本、测试证据和历史说明。截图继续贴合原有纸片背景，取消圆角处理；第 14 页使用个人资料弹窗和统计看板，展示前台与后台的任务差异。

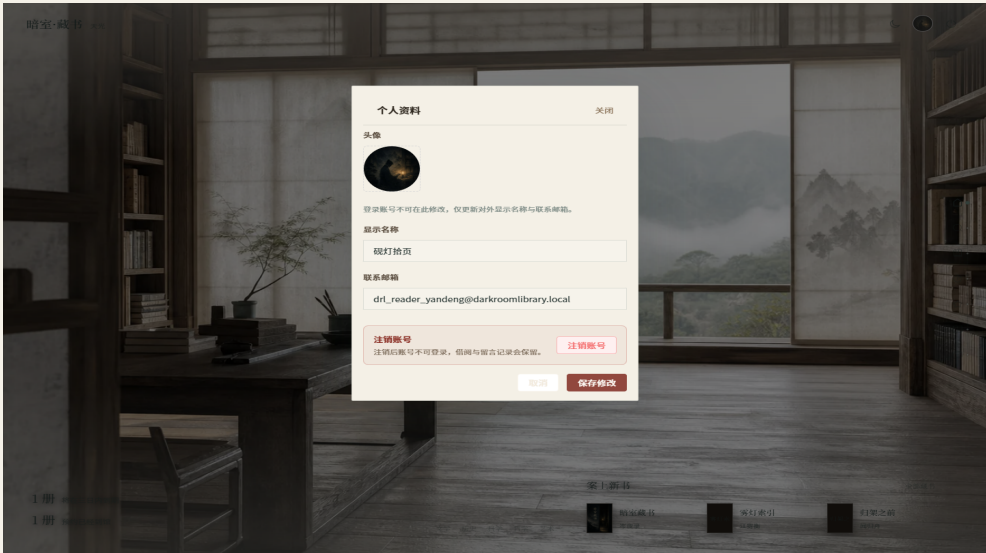


图 4 个人资料弹窗，纸面输入与封蜡红保存按钮。

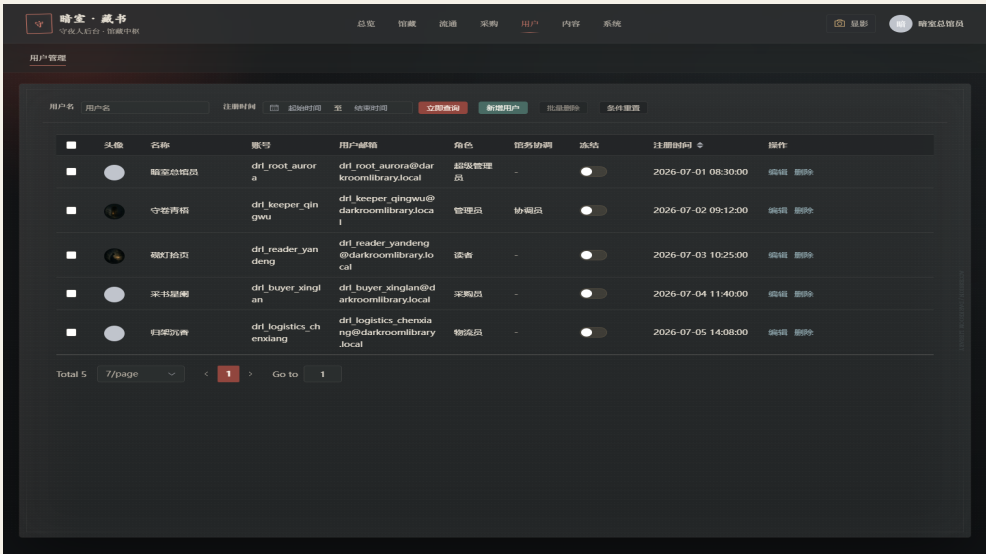


图 5 用户权限管理页面，演示账号为虚构数据。

八、Git 与公开发布

Git 在重构中途加入，因此公开仓库不会伪造课程阶段的提交历史。现有提交从实际初始化时间开始，最终整理只删除临时产物和不应公开的内容，不改写不存在的早期开发记录。提交前执行源码、文档、PPT、PDF、凭据、个人信息和许可证边界扫描；发布包排除 node_modules、target、dist、上传文件、运行证据、日志和本地密钥。

GitHub 发布前检查目标仓库的可见性、分支、发布包和文档链接，确认没有旧压缩包、失效截图、真实账号凭据或本地私密配置。项目源码采用 MIT License，NOTICE 保留项目沿革、第三方依赖和素材边界说明。

公开发布必须同时满足：代码可复现、历史表述诚实、敏感信息已清理、许可证和素材边界清楚。

九、仍然不能声称什么

- 不能把课程落地条件写成项目创意来源，也不能伪造 Git 采用前的提交历史。
- 不能声称已经验证长期生产流量、跨地域容灾或无限规模。
- 不能把本地演示账号当作生产凭据；面向公网部署前必须删除演示账号或逐个改密。
- 不能用一次本地回归替代上线后的监控、备份、恢复和安全响应。
- 不能把模型建议、项目评分或榜单排名当成代码事实。

十、最终结论

这次工程化真正完成的不是给一个 demo 增加更多页面，而是把业务、权限、数据、基础设施、测试和文档放在同一个可验证闭环里。代码可以运行，数据库能够保持一致，浏览器没有运行时异常，公开材料也与当前实现使用同一事实基线。

下一步的重点不再是继续堆叠技术名词，而是保持公开基线可复现：环境变量不入库，文档与代码同步，Git 历史不造假，GitHub 发布前继续检查实际内容。

最终事实基线：后端 226/226，前端 36/36；6 个固定权限身份完成 78 次真实 API；三实例按 96 / 128 / 160 三批执行 1,986 次场景请求，最大 P95 461 ms；116 路由 / 414 次 API / 5,678 次网络响应无浏览器异常；116 个页面和 21 个关键弹层无溢出与越界；27 条外键关系无孤儿和领域违规；npm audit 为 0 vulnerabilities。